

7-2 · 텍스트 데이터 Chunking · 정리·회고 0.5H

정리·회고

분할 문법 한 표, 그리고 7-3 준비

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

30분 뒤에는

- ① 하려는 일과 코드를 한 표로 대응시킨다
- ② 오늘 정한 `chunk_size·overlap` 값을 적어 둔다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 7-3 패키지와 Qdrant 이미지를 미리 받아 둔다

오늘 정한 값은 7-3 에서 그대로 씁니다 — 표와 함께 남겨 두세요

하려는 일과 코드 — 한 표로

하려는 일	코드
텍스트 로드	<code>TextLoader(path, encoding="utf-8").load()</code>
PDF 로드	<code>PyPDFLoader(path).load()</code>
분할기 만들기	<code>RecursiveCharacterTextSplitter(chunk_size=, chunk_overlap=)</code>
분할 실행	<code>splitter.split_documents(docs)</code>
구분자 조정	<code>separators=["\n## ", "\n\n", "\n", " ", ""]</code>
메타데이터 확인	<code>chunk.metadata</code>
메타데이터 추가	<code>chunk.metadata["chunk_id"] = i</code>

PDF 는 페이지당 Document 1개 — 자른 뒤에도 metadata 의 source·page 가 조각마다 따라온다

여덟 개를 스스로 확인해 보세요

- 1 RAG 여섯 단계를 순서대로 말할 수 있다
- 2 문서를 자르는 이유 네 가지를 말할 수 있다
- 3 한글 문서를 `encoding="utf-8"` 지정으로 깨짐 없이 로드했다
- 4 인코딩을 틀리게 줘서 나는 오류를 직접 봤다
- 5 `chunk_size` 와 `chunk_overlap` 의 역할을 각각 설명할 수 있다
- 6 `overlap` 이 실제로 겹치는 것을 이웃 조각 대조로 확인했다
- 7 메타데이터가 조각마다 따라오는 것을 확인하고 직접 추가했다
- 8 파라미터 비교 실험을 근거로 기준값을 정해 기록했다

증상으로 찾는 원인과 해결

증상	원인	해결
UnicodeDecodeError codec can't decode	파일 인코딩과 로더 인코딩이 다름	encoding='utf-8' 명시 · 안 되면 cp949 로 읽기
overlap 을 크게 넣자 ValueError	$\text{overlap} \geq \text{chunk_size}$ — 조각이 못 나아감	size 의 10~20% 로 예: 500 / 50
PDF 로드 후 page_content 가 빈 문자열	스캔 이미지 PDF — 글자 레이어가 없음	드래그로 글자 선택되는지 확인 · OCR 또는 텍스트 원본
Character 분할기에 size 보다 큰 조각	구분자 하나 사이가 chunk_size 보다 김	Recursive 로 교체 — 구분자를 차례로 시도

Windows 에서 만든 한글 파일은 cp949(EUC-KR)인 경우가 많다 — 그래도 실패하면 파일을 UTF-8 로 다시 저장

자주 나오는 질문 네 가지

1 조각 크기의 정답이 있나요?

없다 — 문서 성격과 질문 유형에 따라 다르다. 500~1000 에서 시작해 재 보며 조정하는 것이 현실적

2 표나 코드가 든 문서는?

중간에서 잘리면 망가진다 — MarkdownHeaderTextSplitter 같은 전용 분할기를 쓰거나 전처리로 분리

3 겹치면 같은 내용이 중복 저장되지 않나요?

된다 — 저장 비용이 조금 늘지만 검색 품질을 위해 감수한다

4 이미지가 든 PDF 는?

텍스트 레이어가 없으면 OCR 이 필요 — 이번 과정 범위 밖이지만 실무에서 자주 만나는 문제

다음 차시(7-3)를 위한 준비

지금 받아 두기 — 패키지 · Qdrant 이미지 · 임베딩 키

가상환경	cd ~/data-eng/week7 · source .venv/bin/activate
패키지	pip install qdrant-client langchain-qdrant
이미지	docker pull qdrant/qdrant
문서 확인	ls *.txt *.md — 실습 문서가 있는지
임베딩	OpenAI API 키 또는 ollama pull nomic-embed-text

오늘 정한 chunk_size·overlap 값을 적어 두세요 — 7-3 에서 그대로 씁니다

7-3 학습 목표 — 조각을 숫자로 바꿔 저장하고 찾는다

- ① 임베딩이 텍스트를 어떤 숫자로 바꾸는지, 차원 수의 뜻을 설명한다
- ② 코사인 유사도로 두 문장의 의미적 가까움을 계산한다
- ③ Docker 로 Qdrant 를 띄우고 size·distance 로 컬렉션을 만든다
- ④ 벡터를 업서트하고 query_points 로 의미 검색 결과를 확인한다

단어가 겹치지 않아도 뜻이 비슷하면 찾아내는 검색을 직접 손에 넣습니다

다음 회차 예고

7-3 · Embedding 과 Vector DB

01

이론 1H

임베딩 원리와
유사도 검색

02

실습 1H

임베딩 생성
실습

03

실습 1.5H

Qdrant 컬렉션
생성·적재·검색

04

정리 0.5H

정리·회고

준비: Docker Desktop 실행 · OpenAI API 키 또는 로컬 Ollama(ollama pull nomic-embed-text) · Qdrant 컨테이너는 실습 1단계에서 함께 띄운다

7-2 · 정리·회고 0.5H 마무리

다음 회차는 7-3 Embedding 과 Vector DB

7-2 에서 자른 조각을 숫자(벡터)로 바꿔 Qdrant 에 넣고, 질문과 가까운 조각을 찾아내는 것까지 합니다

✓ 임베딩과 차원 수 · 코사인 유사도

✓ Docker 로 Qdrant 기동 · 컬렉션 생성

✓ 벡터 업서트와 query_points 검색

✓ 뜻이 비슷하면 찾아내는 의미 검색

준비할 것

패키지 2종 · Qdrant 이미지 · 임베딩 키